

Docker

for Devs

From concept to practice with analogies from our world

From zero to container

3 Hands-on Exercises

Cheat Sheet

Table of Contents

01	What is Docker?	2
02	Pre-requisite: understanding Containers and VMs	3
03	What is Docker used for?	4
04	Installing Docker	5
05	Essential Concepts	6
06	Exercise 1 — Hello, Container! [Beginner]	8
07	Exercise 2 — Dockerizing your App [Intermediate]	9
08	Exercise 3 — App + Database together [Challenge]	11
09	Cheat Sheet	13
10	Next Steps	14

How to use this mini book

Read from beginning to end the first time.

Os capítulos foram pensados em sequência: cada um prepara o terreno para o seguinte. Se você pular direto para os exercícios sem ler os conceitos, pode ficar perdida em alguns comandos.

Come back whenever you need to.

The Cheat Sheet was made to stay open while you practice. Keep it on your second monitor or printed next to your computer.

Actually do the exercises.

Reading about Docker teaches the theory. Running the commands is what makes it stick. Open the terminal and execute each step. There are no shortcuts here.

01 • What is Docker?

Docker is an open source platform that allows you to create, distribute and run applications inside **containers**. You know that frustrating situation every dev has been through? You spend hours developing, everything works perfectly on your machine, then you send it to a colleague or deploy to production and... it simply doesn't run.

Docker came to end this once and for all. It packages the application together with absolutely everything it needs to run: libraries, environment variables, dependencies and configurations. That package is the **container**, and it runs the same way everywhere.

>> Imagine you baked an incredible chocolate cake. Instead of just giving your friend the recipe, you hand over the cake already assembled, with all ingredients measured, in the right box, ready to serve. It doesn't matter if her kitchen is different from yours, the cake will turn out the same. That's exactly what Docker does with your application: it delivers the finished cake, not just the recipe.

[DICA] Por que isso importa pra você?

Nowadays applications need to run in many different places: on the dev's computer, on the test server and in production on the cloud. Without Docker, each of these environments would need to be configured by hand. Any tiny difference turns into a mysterious bug that nobody can reproduce. With Docker, everyone runs exactly the same thing, no surprises.

02 • Pre-requisite: Containers and VMs

Before diving headfirst into Docker, it's worth understanding two concepts it builds upon: **virtual machines (VMs)** and **containers**. If you already know both, feel free to skip straight to chapter 03. But if you still have doubts, stay here because this summary will clear everything up.

[LIVRO] Quer aprofundar mais?

O que você vai ler aqui é um resumo introdutório para você não ficar perdida. O Volume 02 da Deploy from Zero Series, sobre Git e GitHub, e materiais de Sistemas Operacionais cobrem esses fundamentos com muito mais profundidade. Vale a leitura para construir uma base mais sólida!

Virtual Machine (VM)

A VM simulates an entire computer inside your computer. It has its own operating system, memory and disk, all virtualized. Think of it this way: it's like living in an apartment inside a building. The building is the real physical server, and the apartment is the VM running inside it.

>> *Imagine a biggest professional makeup case, with all the compartments, built-in mirrors, own lights and even a battery charger. It's complete and self-sufficient, but way too heavy to carry around every day. A VM is like that: works on its own for everything, but consumes a lot of machine resources.*

Container

The container is a much lighter approach. Instead of virtualizing the entire operating system, it shares the kernel — the core of the OS — from the host computer. It isolates only what the application needs, without carrying all the rest.

>> *Now think of a compact makeup pouch you put together for that specific event: just the lipstick, foundation and mascara you'll use. Light, easy to grab and perfect for that moment. The container is exactly that: lean, fast and focused on what matters.*

VM vs Container: quick comparison

Criteria	VM	Container
Size	Gigabytes	Megabytes
Startup	Minutos	Segundos
SO próprio?	Sim	Não (compartilha)
Portability	Média	Alta
RAM Usage	Alto	Baixo

03 · What is Docker used for?

Great, you already know what Docker is. But where does it come into practice? Where will you find it in your daily dev work? Let me show you the most common use cases.

Consistencia entre ambientes

Essa é a dor que o Docker resolve melhor. Sabe o famoso *na minha máquina funciona*? Com o Docker isso deixa de existir. O container garante que o ambiente de desenvolvimento, o de testes e o de produção rodam a mesma coisa, do mesmo jeito, sempre.

Faster deploys

Containers start up in seconds. This completely transforms the process of updating an application in production. Less time waiting, more time delivering value.

Microservices

Each part of the application can run in its own container, with its own version of the language and dependencies, without conflicting with the other parts. It's the ideal model for modern architectures.

>> Pensa num look completo montado pra um evento. O vestido, os brincos, a bolsa e os sapatos podem ter vindo de lojas completamente diferentes, mas você combinou tudo com perfeição. No Docker é igual: cada container é uma peça independente, mas juntos formam a aplicação completa funcionando lindamente.

Portability para a nuvem

A container that works on your machine will work exactly the same on AWS, Azure or Google Cloud. No need to reconfigure anything, no nasty surprises at deploy time.

Isolated test environments

Need to test the application with a different version of the database? Spin up a container, test everything you need and then delete it. Your system stays intact, nothing installed unnecessarily.

[STAT] Docker no mercado

According to the Stack Overflow Developer Survey, Docker is among the most loved and used platforms by devs worldwide. Mastering Docker today is an essential skill for any backend, DevOps or cloud position.

04 · Installing Docker

Before getting our hands dirty, we need to install Docker on your machine. The process is quite smooth and varies slightly depending on your operating system. Choose your case below:

Windows and Mac

A forma mais simples é instalar o **Docker Desktop**, que já vem com interface visual é tudo que você precisa pra começar:

- Go to <https://www.docker.com/products/docker-desktop>
- Download the version for your operating system
- Siga o instalador normalmente ate o final
- Open Docker Desktop and wait for it to fully initialize

Linux (Ubuntu or Debian)

Open the terminal and run the commands below in sequence:

Terminal Linux

```
# Atualizar pacotes
sudo apt-get update
# Instalar o Docker
sudo apt-get install docker.io -y
# Iniciar o servico
sudo systemctl start docker
sudo systemctl enable docker
# Adicionar seu usuario ao grupo docker (evita usar sudo)
sudo usermod -aG docker $USER
```

Verifying the installation

Abra o terminal e rode esses dois comandos. Se tudo estiver ok, você já está pronta pra começar:

Terminal

```
docker --version
# Esperado: Docker version 24.x.x
docker run hello-world
# Se aparecer Hello from Docker! está tudo certo!
```

[!] Atenção usuarias Windows

Make sure WSL 2 (Windows Subsystem for Linux) is enabled on your machine. Docker Desktop itself will guide you through this setup if it's not yet active. It's the only technical prerequisite on Windows and the process is quite simple.

05 · Essential Concepts

Before we get to the hands-on part, let's align on the vocabulary. These are the terms you'll see all the time when working with Docker. You don't need to memorize everything now, but it's worth bookmarking this page to check back whenever you need.

Image

Definição: É o modelo, o molde do container. Ela é imutável e pode ser reutilizada quantas vezes você quiser.

>> Pensa na receita completa do bolo, com todos os ingredientes e o modo de preparo. Você pode usar a mesma receita para fazer varios bolos. A imagem é essa receita e cada bolo que sai dela é um container.

Container

Definição: É a imagem em execução. Uma instância ativa e isolada da sua aplicação rodando na sua máquina.

>> O bolo já assado, montado e pronto pra servir. Você pode ter varios bolos feitos da mesma receita ao mesmo tempo, cada um num pratinho diferente.

Dockerfile

Definição: Um arquivo de texto com as instruções para construir uma imagem. É o ponto de partida de tudo.

>> A sua caderneta de receitas escrita do zero, passo a passo: pre-aqueça o forno, misture os ingredientes, asse por 40 minutos. O Docker le esse arquivo e cria sua imagem seguindo cada instrução.

Docker Hub

Definição: Um repositório público cheio de imagens prontas pra usar. Funciona como um GitHub, mas para imagens Docker.

>> A loja de insumos de confeitaria online. Você não precisa criar tudo do zero: já tem imagem pronta do PostgreSQL, Redis, Nginx é muito mais. É só baixar e usar.

Port Mapping

Definição: Mapeia uma porta de dentro do container para uma porta da sua máquina, deixando a aplicação acessível.

>> *Think of the intercom in your apartment building. The container has its door inside, but you configure the entrance through the building gate for the outside world to reach it.*

Volume

Definição: Mecanismo para persistir dados do container no disco da máquina host.

>> *Like that cloud folder you sync across all your devices. Whatever you save to the volume persists even if the container is removed. Without a volume, when the container dies, the data disappears with it.*

Port mapping examples

Comando	O que faz
-p 8080:80	Access localhost:8080 to reach port 80 of the container
-p 5432:5432	PostgreSQL database accessible on your machine normally
-p 3000:3000	Node.js app running in the container, accessed on port 3000
-p 443:443	Direct HTTPS, no port translation

[DICA] Resumo rápido antes de praticar

Image is the mold. Container is the mold running. Dockerfile is the file that creates the mold. Docker Hub is where the molds are stored. Volume saves your data. Port is the communication channel with the outside world. Now you have the complete vocabulary for the exercises!

Exercício 1 — Ola, Container!

BEGINNER

Objective: Run your first Docker container and understand the basic flow from image to container.

Estimated time: 10 minutes

What you will learn

- How to download an image from Docker Hub
- How to run a container with the **docker run** command
- How to list running and stopped containers
- How to stop and remove containers

Step by step

1. Confirm that Docker is installed and working:

```
docker --version
```

2. Run the hello-world container, Docker's classic Hello World:

```
docker run hello-world
# Docker will do four things in sequence:
# 1. Look for the hello-world image on your machine
# 2. Nao encontrar (e a primeira vez!)
# 3. Download it automatically from Docker Hub
# 4. Create and run the container
```

3. Start an Nginx web server running in the background:

```
docker run -d -p 8080:80 --name meu-nginx nginx
# -d      = runs in background, without blocking the terminal
# -p      = mapeia a porta: sua_máquina:container
# --name   = da um nome amigavel pro container
```

Open your browser at **http://localhost:8080**. The Nginx default page will appear.

4. Practice the management commands:

```
docker ps          # lists currently running containers
docker ps -a       # lista todos, ate os parados
docker stop meu-nginx # stops the container
docker rm meu-nginx  # removes the container
docker images      # lists downloaded images
```

[OK] Resultado esperado

You downloaded an image, started a container, accessed it through the browser and practiced the essential commands. If Nginx appeared in the browser, you completed the exercise successfully!

Exercício 2 — Dockerizando sua App

INTERMEDIATE

Objective: Create a Dockerfile for a Python application and build your own Docker image from scratch.

Estimated time: 20 to 30 minutes

What you will learn

- How to write a **Dockerfile** from scratch
- How to build an image with the **docker build** command
- How to run a container from your own image
- Best practices when writing a Dockerfile

We'll create a small Python API using Flask. It will return a personalized message with your name when accessed through the browser.

Step by step

1. Create a project folder and enter it:

```
mkdir minha-api-docker
cd minha-api-docker
```

2. Crie o arquivo **app.py** com o seguinte conteúdo:

```
app.py
# app.py
from flask import Flask
import os
app = Flask(__name__)
@app.route('/')
def home():
    nome = os.getenv('NOME_DEV', 'Dev')
    return f'Oi, {nome}! Seu container está rodando com sucesso'
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

3. Create the **requirements.txt** file:

```
requirements.txt
flask==3.0.0
```

4. Crie o **Dockerfile** (sem nenhuma extensão no nome):

```
Dockerfile
FROM python:3.11-slim
WORKDIR /app
# Copiamos as dependências primeiro para aproveitar o cache
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ['python', 'app.py']
```

5. Build the image and start the container:

```
# Constroi a imagem (o ponto indica a pasta atual)
docker build -t minha-api:v1 .
# Sobe o container passando seu nome como variável
docker run -d -p 5000:5000 -e NOME_DEV='Gabi' --name api minha-api:v1
```

Access **http://localhost:5000** in the browser and see your personalized message.

[DICA] Boas práticas no Dockerfile

Always copy the dependencies file (requirements.txt or package.json) BEFORE copying the rest of the code. Docker uses a layer caching system: if the dependencies haven't changed, it reuses the already built layer and the build is much faster. It only copies new code when something actually changed.

Comandos uteis durante o desenvolvimento

1. See what's happening inside the container:

```
docker logs -f api          # follows logs in real time
docker exec -it api bash    # opens a terminal inside the container
```

2. Recreate the container after changing the code:

```
docker rm -f api
docker build -t minha-api:v1 .
docker run -d -p 5000:5000 -e NOME_DEV='Gabi' --name api minha-api:v1
```

[OK] Resultado esperado

Your image appears in the docker images listing, the container starts without errors and the browser displays the message with your name. You just created your first custom Docker image!

Exercício 3 — App + Banco com Docker Compose

CHALLENGE

Objective: Orchestrate two containers at the same time, the API and a database, using Docker Compose.

Estimated time: 30 to 40 minutes

[DICA] Spoiler do próximo volume!

Docker Compose is the full topic of Volume 04 of the series. Here you'll get a taste of what it does, enough to start using it day to day. To truly master it, don't miss the next mini book!

What you will learn

- What is the **docker-compose.yml** file and what it's for
- How to connect containers through an internal network
- How to pass environment variables to configure services
- How to bring everything up at once with a single command

We'll expand the API from the previous exercise so it connects to a PostgreSQL database.

Step by step

1. Update the **requirements.txt**:

```
requirements.txt
flask==3.0.0
psycopg2-binary==2.9.9
sqlalchemy==2.0.23
```

2. Create the **docker-compose.yml** file — API block:

docker-compose.yml (parte 1)

```
version: '3.8'
services:
  api:
    build: .
    ports:
      - '5000:5000'
    environment:
      - NOME_DEV=Gabi
      - DB_HOST=banco
    depends_on:
      - banco
    networks:
      - rede-app
```

. Continuation — database and volumes block:

docker-compose.yml (parte 2)

```
banco:
  image: postgres:15-alpine
  environment:
    - POSTGRES_USER=postgres
    - POSTGRES_PASSWORD=minhasenha
    - POSTGRES_DB=appdb
  volumes:
    - dados-banco:/var/lib/postgresql/data
  networks:
    - rede-app
volumes:
  dados-banco:
networks:
  rede-app:
```

3. Bring up the entire application with a single command:

```
docker compose up -d
docker compose logs -f    # follows the logs
docker compose down       # stops everything
```

>> Lembra quando você montou um look completo para aquele evento especial? Vestido, brincos, bolsa e sapato, cada peça no lugar certo, todas combinando entre si. O **docker-compose.yml** faz exatamente isso com a sua aplicação: descreve cada serviço, como eles se conectam e garante que tudo suba junto.

[OK] Resultado esperado

Com um único docker compose up, sua API e o banco sobem juntos e já se enxergam na rede interna. A API fala com o banco usando apenas o nome banco, sem precisar saber o IP de nada. Os dados persistem no volume mesmo que o container seja reiniciado.

09 • Command Cheat Sheet

Pin this page on your wall or save it in your favorites. These are the commands you'll use every week when working with Docker:

Images	
<code>docker images</code>	Lists all local images
<code>docker pull nginx</code>	Downloads an image from Docker Hub
<code>docker build -t nome:tag .</code>	Builds image from the Dockerfile
<code>docker rmi nome:tag</code>	Removes an image
<code>docker image prune</code>	Removes images that are not being used
Containers	
<code>docker run -d -p 8080:80 nginx</code>	Starts container in background with port
<code>docker run -it ubuntu bash</code>	Starts interactive container with terminal
<code>docker ps</code>	Lists currently running containers
<code>docker ps -a</code>	Lists all containers
<code>docker stop id</code>	Stops a container
<code>docker rm id</code>	Removes a stopped container
<code>docker rm -f id</code>	Stops and removes at once
Inspection and Logs	
<code>docker logs id</code>	Views the container logs
<code>docker logs -f id</code>	Follows logs in real time
<code>docker exec -it id bash</code>	Opens terminal inside the container
<code>docker inspect id</code>	Shows all container details
<code>docker stats</code>	Monitors CPU and memory in real time

Cleanup	
<code>docker system prune</code>	Removes everything not in use
<code>docker volume ls</code>	Lists existing volumes
<code>docker volume prune</code>	Removes unused volumes
<code>docker network ls</code>	Lists created networks

10 • Next Steps

Congratulations on making it here! You now have a solid Docker foundation and can already use it in your daily work. But the journey is far from over. Here's what's coming up in the series:

Vol. 04 — Docker Compose

You already got a taste of it in exercise 3. The next mini book dives deep into Compose: managing multiple services, custom networks, healthchecks and how to set up a complete development environment.

Vol. 05 — REST APIs with Python

Learn to build complete APIs using Flask or FastAPI, with REST best practices, authentication and documentation. All dockerized, of course.

Vol. 06 — AWS in Practice

Com Docker dominado, a nuvem fica muito mais tranquila. Vamos ao Elastic Container Service (ECS), ECR para hospedar suas imagens, é muito mais.

Resources to keep learning

- **Official documentation:** docs.docker.com — always the most up-to-date and reliable source
- **Docker Hub:** hub.docker.com — explore the available ready-made images
- **Play with Docker:** labs.play-with-docker.com — practice directly in the browser without installing anything
- **Livro:** Docker Deep Dive de Nigel Poulton — leitura rápida, direta é muito prática

What you mastered in this volume

- What Docker is and why it matters in the market
- The difference between VM and container and when to use each
- The 6 essential concepts: Image, Container, Dockerfile, Docker Hub, Port Mapping and Volume
- How to run containers, download images and manage the lifecycle
- How to write a Dockerfile from scratch and build your own image
- How to orchestrate API + database with Docker Compose

You made it here. That already puts you ahead of a lot of people.

Docker is not just a tool. It's a way of thinking about environments, reproducibility and care for the code you deliver.

Did you enjoy this mini book?

Compartilha com aquela amiga dev que ta começando a jornada!

Tag me on Instagram **@deployada** with your progress.

Adoro acompanhar voces evoluindo!

Instagram	LinkedIn	Newsletter
@deployada	linkedin.com/in/mgcarvalho	deployada

Continue your journey! >>

Vol. 04 • Docker Compose — orchestrating containers

Vol. 05 • APIs REST com Python

Vol. 06 • AWS in practice: S3, EC2 and Lambda

Vol. 07 • Messaging: SQS & SNS

Vol. 08 • Observability: Datadog & Splunk

"Every great dev ran their first container once."

— @deployada

@deployada

Instagram • LinkedIn • Newsletter